

Counting Versus Testing the Destination: A Simulation Study of Layered Defences Against SMS OTP Flooding and Pumping

Naveed Ahmed^{a,*}

^aCollege of Computer and Information Science, Prince Sultan University, Riyadh, Saudi Arabia

Abstract

Endpoints that send one-time passwords via SMS are abused to waste money (flooding) or to earn part of the termination fee (SMS pumping). We evaluate in simulation a twelve-step defense that draws trust from app attestation, signed sessions, and verification outcomes, against simpler designs and 25 attacker profiles. Attacks with a signature in the request are contained at a low cost to simulated users; a residential attacker with farmed CAPTCHA scores is rationed by refusing half of the first-time users. On the destination, short-window counters of SMS sent per number block, selected at a benign service target, leaked 6 to 48 messages against four concentrated 20-minute pumpers at 0.06 to 0.43 points of attack-free completion, where at the busiest density, the default sequential tests of verification outcomes leaked 19 to 452. Under a second predeclared protocol on seeds and workloads held out from selection, the frozen counter met its leakage and benign-cost claims, leaked no more than the tests in all but 13 of 144 cells, all against an instantly verifying carrier, and failed both service-under-attack claims when the pumper shares blocks with real users. For attacks of up to an hour, unfitted estimates whose crossing is algebraic, not a measured switching point, ordered the policies' mean leakage in 18 of 18 decisive boundary cells with the counter's cold-start bound and 17 with its warm-start bound: the counter loses its advantage against paced or widely spread pumpers.

Keywords: SMS pumping, Artificially inflated traffic, One-time password, Rate limiting, Sequential tests, Fraud prevention

1. Introduction

Most online services verify a phone number by sending a code via SMS from an endpoint reachable before any account exists, at a tenth of a cent to twenty cents per message. A flooder makes the service send to numbers it does not own; a pumper sends to a range whose operator shares the termination fee, turning the service's SMS budget into its income (MITRE Corporation, 2026). Twitter's owner said in 2023 that pumping had cost the company approximately \$60 million a year (Commsrisk, 2023), and Huh et al. (2025) found as many attack requests as genuine ones in three months of one large service's logs.

We study the defenses of a registration flow: a baseline of seven controls (v1) and a twelve-step redesign with attested platforms, signed sessions, number checks before a message is paid for, reputation from whether codes were entered, a five-response risk score, and a hard budget. We ask in simulation which attacks it contains against simpler designs, at what cost in completed registrations (RQ1); which layer is responsible (RQ2); when outcome-based reputation separates an attacker from simulated users (RQ3); what rationing costs when it does not (RQ4); and what destination policies cost legitimate users, with and without an attack (RQ5).

Because a pumper is paid only on its partner's ranges, the study focuses on the destination: under a predeclared compari-

son, a plain counter of sends per number block leaked less than our sequential tests of code outcomes, and a second predeclared protocol mapped where that holds and fails. This paper makes the following contributions:

- an analysis of eight gaps in the baseline, a twelve-step design, and an open implementation;
- measurements of where outcome-based reputation stops separating an attacker from users, and of what rationing then costs and depends on;
- two predeclared comparisons of destination policies, a send counter against sequential tests of code outcomes, with their failures and collateral damage;
- unfitted estimates of when counting leaks less than our tests, checked against the measurements.

Section 2 reviews related work; Sections 3–5 give the baseline and threat model, the design with its leakage estimates, and the methodology; Section 6 reports the results; Sections 7–9 discuss them, state the limitations, and conclude.

2. Related work

2.1. Revenue-share fraud

In international revenue-share fraud, traffic is generated to an advertised number range, and the termination fee is split along the route (Sahin et al., 2017). The 3.14 million test numbers

*Corresponding author.

Email address: nahmed@psu.edu.sa (Naveed Ahmed)

from twelve providers collected by [Sahin and Francillon \(2021\)](#) show that the range is the fraudster’s managed asset.

2.2. Detecting artificially inflated SMS

[Huh et al. \(2025\)](#) trained gradient-boosted trees on 9.4 million SMS authentication logs from nineteen countries, reaching 89.6% recall at 0.19% false positives on web traffic. Approximately 73% of web attacks sent four or fewer requests per identity a day, defeating per-entity limits, and destination signals were the strongest: about 92% zero conversion for attacks against 21% for genuine traffic. Their detector requires labeled traffic and acts per request. Their data show that destination aggregation is informative; we add a controlled comparison of two label-free ways to act on it per block, from send counts or code outcomes, and of their cost to users sharing a block. Theirs was not reproduced, so the two are not ranked. [Zalisky and Odarchenko \(2026\)](#) count A2P sends per prefix (a number less its last three digits) in fixed or rolling five-minute windows and quarantine a prefix for 24 hours past a day or night threshold. On metadata of two confirmed incidents fixed windows blocked 95% and 63% of the traffic, and 58% of a legitimate marketing campaign; counting unique recipients blocked 2% of the campaign and none of the first incident. Theirs is a send counter judged on real traffic by messages blocked; ours compares a counter with outcome tests at a service target, in simulation, by what users sharing a block lose; the numbers are not comparable.

2.3. Defenses in practice

Twilio’s Fraud Guard blocks regions known for pumping ([Twilio Inc., 2026c](#)), and its SMS Pumping Risk Score advises friction from 60 and refusal above 90 ([Twilio Inc., 2026a](#)), so graded destination action is established; no product was benchmarked. CAPTCHA solving is a labor market at approximately \$1 per thousand ([Motoyama et al., 2010](#)), open where machine solvers fail ([Dionysiou and Athanasopoulos, 2020](#)); client puzzles ([Dean and Stubblefield, 2001](#)) were not evaluated. Residential proxy addresses rotate within minutes ([Mi et al., 2019](#)), and carrier-grade NAT makes address reputation on cellular networks “all but infeasible” ([Richter et al., 2016](#)).

2.4. Identity and change detection

One entity can present many identities ([Douceur, 2002](#)), and a signed session provides continuity, not scarcity. Our block tests are Wald’s sequential tests ([Wald, 1945](#)) run as Page’s cumulative sums ([Page, 1954](#)). Quickest change detection minimizes delay under false-alarm constraints ([Veeravalli and Banerjee, 2012](#)), and a CUSUM’s parameters follow from the error rates it must meet ([NIST/SEMATECH, 2026](#)); windowed statistics and confidence sequences ([Howard et al., 2021](#)) were not evaluated, so ‘testing’ below means our conversion and speed tests only. Step 7 resembles risk-based authentication ([Wiefling et al., 2019](#)), and attacks on the code once sent are a separate problem ([Mulliner et al., 2013](#); [Siadati et al., 2017](#)).

Table 1

v1 Assumptions and how they fail.

Gap	v1 assumes	How it fails
A	A platform header says which app sent the request	Sending the header reaches every exemption
B	Five requests per address per minute bound an attacker	A fresh residential address per request
C	A number given is worth sending to	Generated numbers cost as much as real ones
D	An allowed country is a safe destination	Premium ranges inside it cost far more
E	Per-key caps bound total spend	A wide attack under every cap
F	Check-then-increment is a rate limit	Two racing instances both pass
G	Bulk and test flags are internal	Neither needs stronger authentication
H	Each check says which check failed	Thresholds can be probed one at a time

3. Baseline and threat model

3.1. The v1 baseline

The baseline, v1, sends a code to any unregistered number behind seven controls: reCAPTCHA v3 on web requests only, one message per number per minute, five requests per address per minute, a country allow-list, a body-length limit, a proxy block, and per-platform caps; bulk and test paths skip some checks. Table 1 lists eight assumptions behind these rules and how an attacker within the threat model breaks each; the gaps follow from the rules, not observed abuse.

3.2. Threat model

The attacker wants as many messages sent as possible, or sent to numbers that pay it. It buys residential proxies and human-like reCAPTCHA tokens (about 0.3 cents each), mints fingerprints, generates numbers, and knows the thresholds. A pumper also has a colluding operator’s ranges, where it may enter codes, report deliveries as failed, build verified history, spread over blocks, and buy challenge solutions. It cannot forge an attestation, use ranges its partner does not terminate, or make a stranger enter a code.

4. Design

No message is paid for before every check passes, and cheap checks run first. Trust comes from evidence the client cannot forge, an attestation, a server-signed session, and the fate of earlier codes, though such evidence can be manufactured (Section 6.8).

4.1. The pipeline and the risk score

A request passes the steps of Table 2, which lists in brackets the gaps of Table 1 each step addresses (FB: the feedback loop), until it is refused. Step 0 takes the platform only from a valid App Attest or Play Integrity result. Each limit and reservation is one atomic Redis operation, but the enclosing steps are not one transaction: on the normal path, a refused request returns

Table 2

Twelve Steps and the v1 Gaps They Address.

Step	Check
0	Proxy block; platform from attestation, never from the header [A]
1	Signed session bound to a fingerprint; 3 requests per session per 10 min [B]
2	Limits per address, /24 and ASN, lower for hosting ASNs [B, F]
3–4	reCAPTCHA v3 threshold and origin check, web only
5	Number intelligence: cost class, sequential numbers, liveness; reads the block’s verdict stage [C, D, G]
6	Body length, excluded numbers; test bypass needs a credential [G]
7	Risk score and tier: allow, delay, challenge, downgrade, block [H]
8	Per-number backoff, 60 s doubling to 1 h, five a day
9	Per-source, platform and country caps with an adaptive multiplier
10	Operating mode: elevated above 80% of the hourly budget, emergency at 100% [E]
11	Reserve hourly count, spend and any destination slot; write state; enqueue; uniform response [E, H]
FB	Outcomes per key; destination-block tests; denylist of fine keys

its per-number claims and counter slot, and the hourly budget is reserved only for a message about to be enqueued, a reservation rather than a completed dispatch; a crash between them can leak reservations (Section 5). Address limits and source caps count requests, except that a request Step 11 challenges returns its source-cap count. Every outcome after Step 0, except the challenge, returns one HTTP 200 body after at least 400 ms, a floor that cannot hide longer work.

Step 7 computes

$$s = \sum_i w_i x_i + w_c \frac{\max(0, \theta - r_{\min})}{\theta} + w_s f + w_m i - w_v v_n - w_f v_f, \quad (1)$$

where the x_i are request signals (CAPTCHA score, hosting ASN, fingerprint age, session requests, country mismatch, number class), $r_{\min}$ is the lowest conversion ratio over the request’s keys (address to destination block), $\theta = 0.3$, $w_c = 25$, f and i flag a flooded key and one whose codes are entered within seconds, and v_n , v_f mark a number and fingerprint with verified history; other weights are 5 to 25 (Ahmed, 2026). Tiers start at 20, 40, 60, and 80: *allow*, *delay* (a doubling wait), *challenge* (−20 when solved), *downgrade* (push or WhatsApp), and *block*.

4.2. The feedback loop

A send resolves as *verified* when its code is entered, *fails* after five incorrect attempts or two minutes past the delivery receipt, and is *undelivered* without a receipt in 60 s or with a failed receipt; only the first two count toward reputation. A positive receipt after a negative one reopens the send; a late verification reclassifies it, but an issued verdict stands. Each transition records its effects and its time in the same compare-and-set, and a recovery sweep completes the work of a dead process; each effect type applies at most once, and exactly once if the sweep runs within 20 minutes. Reputation counts land in the transition’s hour, and a block keeps its latest outage failure time; however, a block’s statistic accumulates in processing order, and a verdict is dated at the event that completes the crossing; thus, a late event can backdate a verdict that contains later evidence, moving its expiry and the next test’s start (by 10 min in one diagnostic). Simulated runs apply events in time order, but concurrent live instances may not, because a transition is

committed before its effects apply; thus, the same dependence holds live unless a block’s effects are serialized.

4.3. Destination policies

A pumper is confined to its partner’s ranges but can rotate numbers and blocks within them. Destination policies are keyed on the 8-digit block (10 000 numbers); therefore, protection weakens as a pumper spreads.

4.3.1. Sequential tests

The conversion statistic assumes that users verify with $p_\ell = 0.8$ and a flooder with $p_a = 0.1$: a failure adds $\ln \frac{1-p_a}{1-p_\ell} = 1.50$, a verification $\ln \frac{p_a}{p_\ell} = -2.08$. The speed statistic assumes that a machine enters a verified code within five seconds with $q_a = 0.9$ and a person with $q_\ell = 0.2$, because autofill removes the keystroke (Google LLC, 2026; Web Incubator Community Group, 2026); it adds 1.50 or −2.08. A verdict is issued, and the statistic is reset when either exceeds $h = \ln 1000 = 6.91$. The floor $S_t = \max(-ch, S_{t-1} + \text{increment})$ sets the memory: unbounded c is Wald’s test, $c = 0$ Page’s cumulative sum; the default is $c = 1$. A first verdict makes the block’s first-time clients pass a challenge for an hour; a second within the hour moves them off SMS, refusing those without another channel.

4.3.2. Counters

A counter limits the number of SMS sent per block in a daily or short refilling window. Step 11 reserves a slot and decides the stage in one atomic operation when an SMS is about to be sent, so concurrent requests cannot skip the challenge. Past the limit, a graded counter challenges first-time clients, and past twice the limit moves them off SMS; a refusing counter refuses. Clients with verified history are exempt from both policies; therefore, a code entered on a pumper’s number can buy that number an exemption. A token bucket of the same rate (burst 4 or 12; its challenge-tier bucket refills independently, unlike RFC 2697’s (Heinänen and Guerin, 1999)) and the counter combined with the tests were also explored.

4.4. Capacity controls

Step 9 multiplies its caps by 0.25 to 1.5 according to a baseline job comparing each key with its expected rate, learned from

the pipeline’s hourly counters; without history, the static caps stands. Verified history avoids adaptive reduction, not the static caps or the budget, which bounds SMS count and spend, not vendor or fallback costs.

4.5. Leakage estimates and a heuristic crossover

We compare the SMS sent to a pumper’s number over an attack of $T \leq 60$ minutes, by a pumper that solves no challenge and holds no verified history. Against the sequential tests, one reaching B blocks with N requests at rate λ leaks about

$$L \approx \min(N, kB + \lambda\tau), \quad (2)$$

before its first verdict, where $k = \lceil h/1.50 \rceil = 5$ and τ , the time a send stays unresolved, is the resolution timeout plus the delivery delay plus half the timeout worker’s period (2.55 minutes) if the carrier never verifies, and seconds if it verifies at once. Judged, such a pumper is held for the verdict’s hour; therefore, for $T \leq 60$ Equation (2) also estimates the attack’s total, ignoring prior credit and outage suspension. A carrier verifying a share σ of codes with a human-like delay gives the conversion statistic an expected increment of $\sigma(-2.08) + (1-\sigma)(1.50)$, zero at $\sigma^* = 0.42$; above it, the statistic drifts away from a verdict.

A counter of q sends per block opens a window with the block’s first reserved SMS and closes it after the window length. A pumper that finds its blocks’ counters empty (a cold start) sends at most

$$L_c \leq \min(N, qBw), \quad w = \lceil T/\text{window} \rceil, \quad (3)$$

whatever its carrier does with the codes. If a legitimate send has already opened a window on block b , using $c_b \geq 1$ slots, the pumper can also take the remaining $(q - c_b)^+$ before at most w further windows open, so in general

$$\begin{aligned} L_c &\leq \min(N, qBw + \sum_{b \text{ open}} (q - c_b)^+) \\ &\leq \min(N, B(qw + q - 1)). \end{aligned} \quad (4)$$

Solving challenges doubles these bounds, and a verified history exempts a client from them. While neither policy’s estimate reaches N , the cold-start bound lies below the tests’ estimate when

$$B(qw - k) < \lambda\tau, \quad (5)$$

and a warm start adds the residual quota to the left side. Equation (5) is the algebraic crossing of two simplified quantities, neither of which is an empirically established switching point or a necessary condition: other limits can hold either policy below its estimate. A carrier that games outcomes may never be judged, so the tests’ leakage approaches N .

In the code, every send takes a slot, verified clients’ included. If a block’s sends arrive as a Poisson stream with mean m per window and all reach the counter, a window holds $1+X$ of them, $X \sim \text{Poisson}(m)$, and the idealized share beyond the quota is $E[(1+X-q)^+]/(1+m)$: 7.3% at $m = q/2$ and 27% at $m = q$ for $q = 4$. This is not a challenge rate: exempt clients take slots without challenge, and challenge failures, the second tier and earlier gates are ignored. The estimates use only the rules, configuration, and attack rate; what an excess send costs a user needs the simulation.

5. Methodology

5.1. Implementation

Approximately 4 000 lines of Python implement the pipeline on in-memory and Redis backends. Up to release 2.8.3, the simulator’s in-memory store let a sorted set expire a fixed time after its first write, whereas Redis kept it while written to; therefore, the simulated outage detector and Step 5 lost recent observations. Release 2.9.0 follows Redis in every rule compared, checked by differential tests, and every study was rerun with it; earlier releases repaired seven replay and concurrency defects, each with a regression test. Of 453 tests, 19 used a real redis-server, 18 tested specific interleavings of two instances, not all. Step 11 was still not one transaction: a crash after its reservations leaked reserved slots or resolved the send as undelivered. The implementation was a research prototype.

5.2. Simulation

The pipeline runs against scripted vendor fakes on a simulated clock. After a warm-up of ten minutes (30 or 130 for pumping), attacks run over legitimate traffic at 20 requests per minute. Simulated users verify 80% of codes after a lognormal delay (median 22 s; 2 s for the 20% using autofill); 70% of numbers are reachable on WhatsApp; 90% of users complete a challenge; 20% are returning account holders with a stable fingerprint, a trusted number, and verified history; thus, the results for returning users describe trusted identities only. Separate random streams provide all scenarios at the same seed with the same users.

Three behavioral parameters have published counterparts: genuine conversion (one provider’s 68% or more (Twilio Inc., 2026b); ours 60%–90%) and the never-converting destinations and identity churn of Section 2; the rest are assumptions. Table 3 groups the attacker profiles by capability and marks the destination policies each faced (S, sequential tests; C, counters; B, block-limit; parameters in results/attacker_profiles.md). Attackers draw pool size, rate, and CAPTCHA class per seed, which explains 92% [80, 98] of the variance of the farm’s leakage in a nested design (ten configurations, three seeds each). Brackets in the text and tables, and bars and bands in the figures, are 95% percentile-bootstrap intervals over seeds, paired per seed for differences; they describe variation under this generator, not uncertainty about real traffic.

5.3. Two protocols and their chronology

A supplementary map (results/study_map.md) gives each study’s seeds. The first protocol, committed before its runs, fixed the matched comparison and a robustness study over 12 Latin-hypercube points (conversion 0.6–0.9 and five other behavioral parameters) and a held-out family (60 to 120 attack requests a minute, 50 000 to 500 000 addresses, heterogeneous legitimate traffic), with claims C1–C5 that hold if true in 90% of cells, a benchmark convention, not a confidence level. The matched comparison changes only the destination policy and, at three block densities, selects on tuning seeds the least leakage summed over four concentrated pumpers among settings

Table 3

Attacker Profiles, Capabilities, and Policies.

Capability	Profiles	Policies
A signature in the request (5)	One address; datacenter rotation abroad; a premium-rate prefix; numbers walked upward; the farm with an unattested iOS header	S, B
Looks like a person (7)	Residential bot; farmed CAPTCHA with fresh or aged fingerprints; one 48-hour-old browser profile	S, B
Games outcomes or trust (8)	Low and slow, 2 or 10 requests/min over 20 ASNs; datacenter rotation buying challenge solutions	S
	Pumpers on elevated-cost or standard ranges verifying within 1 s, or 60% after 30 s	S
	Threshold-aware and receipt-faking carriers; three trust builders (500 identity/number pairs verify for 10 or 30 min, then flood)	S, C
Concentrated on the destination (5)	Four pumpers on three blocks (carrier never verifies, verifies within 1 s or 60% after 30 s, or buys challenges); a poisoner on users' blocks	S, C
Spread (generated)	3 to 300 blocks, at full rate or at most the counter's quota per block	S, C

at or above the *tuning target*, the default's completion minus 0.5 points. The robustness study's first run failed C1 in 32 of 108 cells because attack and no-attack runs drew different users; we fixed the pairing, a fix that followed a result, and reran every study. The second protocol (claims K1–K5) was written after the matched comparison had selected three counters and committed before its runs; six of its seed values had served other robustness points, never selection. Both were rerun unchanged with 2.9.0 (9 919, 7 925 and 240 runs, each carrying its code hash); against 2.8.0, total leakage changed in 855 of 17 964 comparable runs, but the principal selections, eligibility decisions, and claim counts did not; one auxiliary setting (false alarms matched at unbounded credit) moved from *T*1000 to *T*300 under both protocols (artifact change report).

5.4. Designs and metrics

Seven designs: **v1**, **v1-corrected** (attested platform), **budget-only** (sessions, static caps, budget), **block-limit** (a refusing counter of five sends per block per day), **conversion-only** and **speed-only** (v2 minus one block test), and **v2**. A seed is *contained* when leakage stays at most 5% of the attack rate from some minute to the window's end, for five minutes or more. *Attributable* loss compares the same requests with the policy enforced and absent, net of the reverse, in points of offered users: on attack-free traffic it is the *benign cost*, on the attacked trace the *extra harm* relative to the attacked no-policy arm. *Degradation* compares the attacked trace with attack-free operation under the same policy.

6. Results

6.1. RQ1: what is contained, and at what cost

Table 4 gives the leaked share (% , mean of per-run shares) of about 600 attack requests in 20 minutes over 30 seeds under v1, block-limit (BL) and v2 with source caps lifted, and under v2 with caps on, with the first-time users refused and all users completing (%); supplementary Table S1 (results/design_table.md) gives every design with intervals. Datacenter rotation leaks nothing under v2 because it scores in the challenge tier and declines challenges; Step 5 refuses the premium range, and the session cap holds the single client to two messages (Table 4); in all 108 robustness cells, these attacks leaked under 5% at completion within two points of the paired

Table 4

Leakage and User Impact Across Designs.

Attacker	caps lifted: leaked			caps on, v2		
	v1	BL	v2	leaked	refused	completed
Single client	5.6	0.3	0.3	0.3	1.1	79.1
Datacenter rotation	69.0	69.0	0.0	0.0	1.1	79.1
Premium pumping	99.4	1.0	0.0	0.0	1.1	79.1
Sequential numbers	99.4	1.0	11.5	10.7	3.5	77.5
Reused profile	99.5	99.5	22.5	17.4	8.6	74.2
Residential bot	37.2	37.2	37.2	29.9	16.2	69.6
Farmed CAPTCHA	99.5	99.5	99.5	45.6	54.0	45.5
Aged fingerprints	99.5	99.5	99.5	45.6	54.0	45.5
Forged header	100.0	99.5	99.5	45.6	54.0	45.5

no-attack run (claim C1). The walk leaks 61 [53, 70] messages before its block's verdict (block-limit: five), and the reused profile 89 [83, 95].

The farm leaks 99.5% with caps lifted under every design; v2's adaptive cap holds it to 229 [215, 244] messages, 46%, by refusing 54% [47, 60] of first-time users, so 45.5% complete instead of 76.4% under v1: rationing (claim C5 held in 36 of 36 cells).

6.2. RQ2: conditional effects of each layer

In the ablation grid (90 effects, each conditional on all other layers, unadjusted intervals; artifact), removing the adaptive caps lets the farm leak 310 [194, 421] more messages; the risk engine, feedback loop, and block key hold the walk by 166, 152, and 121 messages, and the loop holds the reused profile by 138. The key acts only through the loop (interaction −121 [−141, −101]).

6.3. RQ3: where reputation stops separating

A farmed-CAPTCHA attacker uses an address and fingerprint once and shares its country, prefix, and residential ASN with every legitimate user; thus, its failures penalize everyone: over an hour with caps lifted (five seeds), up to twice the legitimate rate, 99 to 100% of it leaks, since a shared key stays above 0.3 conversion until failures outnumber legitimate sends about 1.7 to one. At ten times, 70% leaks while 17% of legitimate users are challenged and 3.2% refused (claim C2, at least 90% leaked with caps lifted, held in 34 of 36 cells). No single score change separates better: a fresh-fingerprint weight of 40 stops it only by challenging 47% of legitimate users. In the

Table 5

Containment Results for Concentrated Pumping Attacks.

Carrier, design	Leaked (msgs)	Contained (seeds)	Containment (min)	First verdict (min)	Codes entered
never verifies: v1	570 [419, 736]	0/10	–	–	0
v2 default	90 [71, 114]	10/10	3.5 [3.1, 3.9]	3.0	0
verifies within 1 s: v2 default	19 [17, 21]	10/10	1.4 [1.1, 1.7]	0.4	19
verifies 60% after 30 s: v2 default	452 [365, 539]	0/10	–	11.6	272
v2 default, 60 min	1 067 [839, 1 310]	2/10	47.0 [42.0, 52.0]	25.5	–
buys challenges: v2 default	106 [87, 127]	10/10	4.3 [3.8, 4.9]	3.0	0

Table 6

Leakage and Benign-User Impact Across Carrier Types.

Setting (selected as)	NV	IV	HL	CB	Compl.	Benign	Chall.	Events
none	319	566	570	405	64.13%	–	1.53%	0
sequential T1000, c1 (default)	90	19	452	106	64.12%	0.01	1.64%	6.5
sequential T300, c0 (selected at tuning target)	88	14	126	98	63.77%	0.37	6.54%	334.3
counter, graded, 4 per 10 min (selected)	24	24	24	48	64.07%	0.06	2.38%	0
counter, graded, 160 per day (selected daily)	319	409	408	405	64.11%	0.02	1.87%	0

settings run, reputation separated an attacker only on keys its sends dominated (Douceur, 2002).

6.4. RQ4: what rationing costs, and what it depends on

When separation fails, the cap is the lever: over ten seeds, the default lets 47% of the farm’s requests through and refuses 41% of all legitimate users, the tightest setting 20% and 64%. Both assume an oracle job that is told the legitimate rate and runs every minute; running every five minutes or hourly, it lets 75%–96% through. Learning the rate from the pipeline’s counters, the deployed job leaks 193 [161, 229] messages with a clean three-week profile, 340 [239, 441] more on a cold start, 106 [62, 151] more with a stale profile, and 45 [10, 77] more with a poisoned one.

6.5. Pumping against the sequential tests

Table 5 covers concentrated pumpers on three blocks with caps lifted over ten seeds: messages leaked of about 574 requests in 20 minutes, seeds contained, mean minutes to containment (contained seeds) and to the first verdict (seeds with one), and codes entered. The speed test catches a carrier verifying within a second at 19 messages, while the carrier verifying 60% with human-like delay is contained in none of the ten 20-minute runs: its first verdict comes at minute 11.6 on average and 452 messages leak; in an hour, 1 067 leak and two of ten runs are contained, at minute 47.0 on average. Claim C3 (containment with at most 30% leaked) held in 36 of 36 cells.

Across 18 spread configurations, leakage follows the blocks touched: with 30 blocks, five to seven runs in ten are still contained. Unsaturated, Equation (2) errs by 1.6 to 7.9 messages per run (1%–13%, ten configurations); saturated, it overstates by up to 91.

6.6. Destination policies: where the counter wins, and where it fails

6.6.1. At a benign service target

Table 6 reports the first protocol at 200 blocks (144 legitimate sends per block a day; tuning seeds 100–102, evaluation seeds

0–9): 20-minute leakage against the never-verifying (NV), instant (IV), human-like (HL) and challenge-buying (CB) carriers and, from 24-hour attack-free runs at 65% conversion, completion, benign cost (points), users challenged and verdict events a day; the tuning target is 63.95%. Figure 1 plots each setting’s completion against its leakage summed over the four pumpers’ runs of a seed (not one combined attack), with the tuning target dotted; its unlabeled points are sequential settings matched to the default’s false alarms at other credits (key: `results/figure1_key.md`). Figure 2 gives each selected setting’s leakage minus the default’s, paired per seed (negative: leaks less). The sequential setting selected at the tuning target holds the human-like carrier to 126 messages only by raising 334 false verdict events a day, and misses the target by 0.18 points on seeds 0–9. The graded counter of four sends per block per ten minutes leaks 24 against each outcome-reading carrier and 48 against the challenge buyer, at 0.06 points; counters selected at the sparser settings leak 6 to 18 at 0.15 to 0.35 points (up to 0.43 at 80%). With the full grid and density-specific attacks (E5) the rule selects the same counters, and so do other objectives on the tuning runs (worst-case leakage, a first-time-loss constraint, each attacker alone, 2 000 random mixes), except the instant verifier alone and 3 mixes at 200 blocks; on fresh seeds the counter leaked least of the winners against every attacker but the instant verifier (25, against 15 and 16). On seeds 300–309 the 1 000-block counter falls 0.12 points below its target, yet no seed-and-conversion cell fails the paired check (the evaluation default minus 0.5 points).

6.6.2. Under a second protocol

Table 7 covers 12 shifted points of the held-out family, with held-out attack rates and pools, three seeds each and a 20-minute pumper on three blocks real users share; a claim holds if true in 90% of cells, which share workloads and seeds and are not independent, and IV marks the instant verifier’s cells. Frozen and run on this family, the counter leaked 33, 34, 33, and 62 messages against the four carriers, where the default

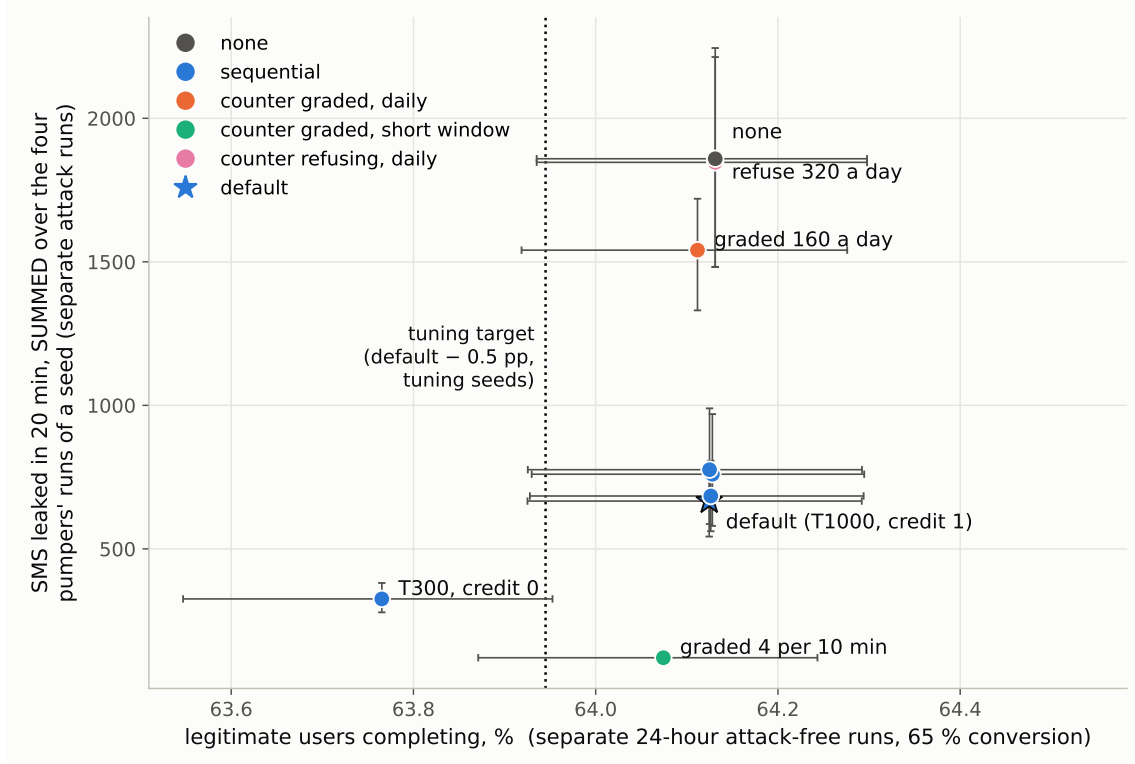


Fig. 1. Leakage vs. Legitimate-User Completion Across Settings.

Table 7

Held-Out Evaluation of the Principal Counter.

Claim	Statement	True / cells	IV	Result
K1	Leaks at most 10% of attack requests	144/144 (100%)	36/36	holds
K2	Attack-free attributable loss at most 0.5 points	36/36 (100%)	–	holds
K3	Attributable loss under attack at most 0.5 points	127/144 (88%)	32/36	fails
K4	Leaks no more than the default sequential tests	131/144 (91%)	23/36	holds pooled, not for IV
K5	Attributable loss under attack at most the default’s plus 0.25 points	93/144 (65%)	23/36	fails

leaked 287, 36, 344, and 287, and no policy 451 to 1 857, at a benign cost of 0.13 [0.09, 0.17] points, and failed both service-under-attack claims: 13 of the 17 cells that broke K3 lie at the three points with the most legitimate traffic, where users and pumper compete for one quota; K5 failed in 51 cells over 11 of 12 points. The counters selected for sparser traffic cost 4.7 and 5.8 points here even without an attack: the quota must match the traffic.

Tables 8 and 9 report E1 at 200 blocks (60-minute attacks, seeds 300–309): a pumper on three blocks real users share, and a poisoner flooding all 200 blocks. Leakage under the pumper is summed over the four carriers’ runs of a seed, which can hide a carrier on which a policy loses (per carrier: artifact); harm is net completions lost in points, averaged over the carriers (negative: a gain), with intervals for leakage and the attacked blocks in Table 8 and for all users’ extra harm in Table 9. In Table 8, against the attacked no-policy arm, the counter cost all users nothing on average and gained the attacked blocks’ users three points on average, since without a destination policy, the pumper exhausts their blocks and the hourly budget; with the human-like carrier alone, it cost them 1.9. Against the attack-free operation,

every evaluated policy left those users 5 to 9 points worse off on average (2 to 12 per carrier; 8 with none): none shields a block an attacker shares. Counter and tests together, explored outside the frozen claims, leaked least summed over the four carriers but not against the instant verifier (28; T300, c0: 22), and under the poisoner (Table 9) cost all users most of the counters and buckets and leaked more than T300, c0. A token bucket with a burst of 4 spared the poisoner’s users only by letting almost everything through; with two untuned settings, neither limiter dominated, and we ranked only the implemented policies. Returning users, all simulated with verified history, lost nothing.

6.6.3. The operating boundary and adaptive attackers

Figure 3 shows the counter’s operating boundary (E4, seeds 300–304): in (a), the attributable loss of the three hot blocks’ users against their legitimate rate, attack-free and with a never-verifying pumper on those blocks; in (b, c), leakage in 20 and 60 minutes against the blocks a pumper uses, at full rate or paced, with Equations (2)–(4) for the full-rate pumper as hollow markers. All horizontal axes and the vertical axes of (b, c) are logarithmic, and lines join sampled settings without locat-

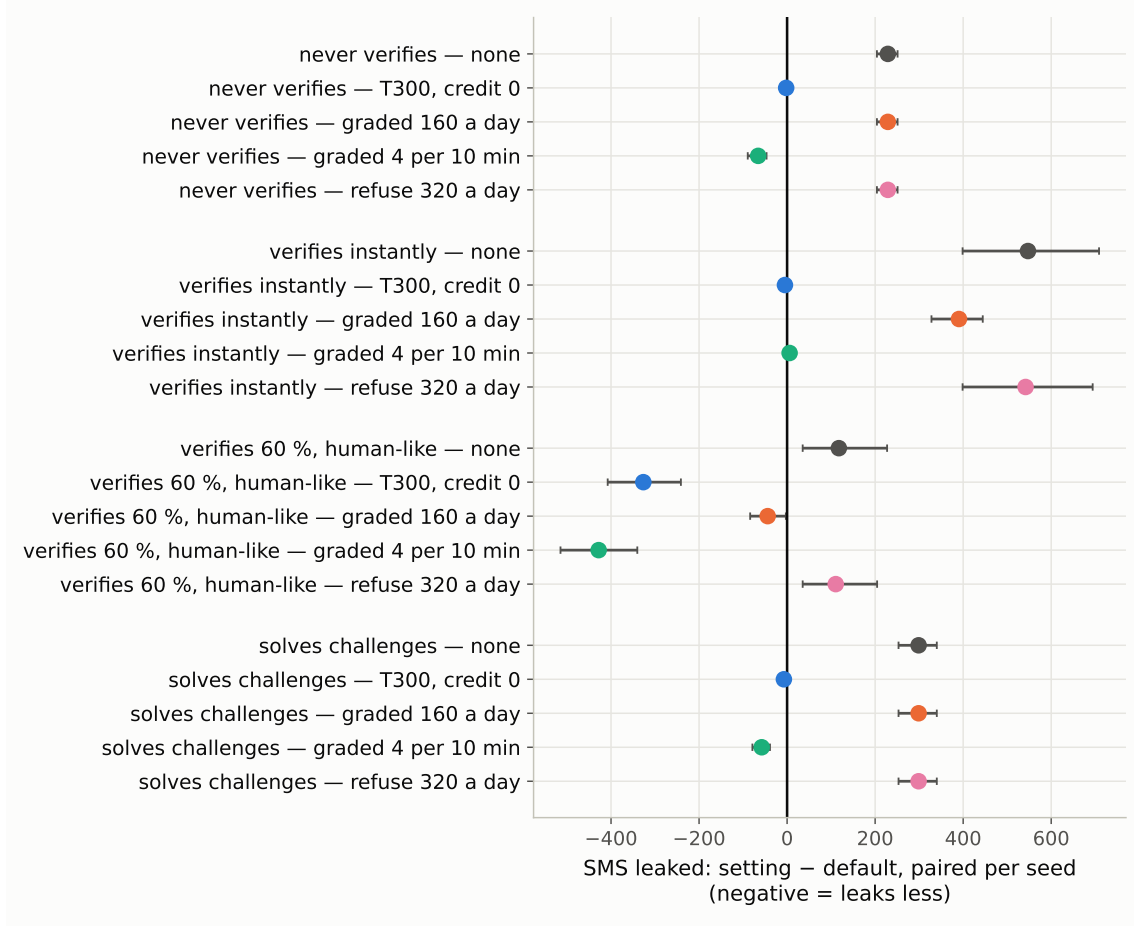


Fig. 2. Leakage Difference from the Default Across Attack Types.

ing a crossover. Attack-free, the counter costs a block’s users nothing at one send per ten minutes, 2.3 points at four and 5.5 at eight, far below the idealized share beyond the quota (27% at four), since most challenged users complete or switch to WhatsApp; without WhatsApp, eight sends cost 13.2. A 30-minute

launch at three times the normal rate costs its blocks’ users 16 points. Over 60 minutes (Figure 3c) the default leaked 110, 144 and 242 messages on 3, 10 and 30 blocks, against 105, 139 and 240 from Equation (2) ($\lambda\tau \approx 90$), and the counter 71, 240 and 679 against its cold-start bound of 72, 240 and 720. Once 7

Table 8

Leakage and User Harm Under Shared-Block Pumping.

Policy	Leaked (SMS)	Extra harm against attacked, no policy		Degradation against attack-free, same policy		Benign cost all users
		all users (≈ 1200)	attacked blocks (≈ 18)	all users	attacked blocks	
none	5 509	–	–	0.19	8.0	–
	[4 409, 6 567]				[5.1, 10.7]	
sequential, default	1 149	–0.07	–1.0	0.10	7.0	0.02
	[953, 1 340]		[–2.8, 0.8]		[4.7, 9.1]	
sequential T300, c0	397	0.13	0.9	0.12	8.9	0.20
	[312, 481]		[–1.3, 3.3]		[6.0, 11.7]	
counter, 4 per 10 min	403	–0.05	–3.2	0.07	4.8	0.07
	[386, 418]		[–5.0, –1.1]		[2.9, 6.7]	
counter + sequential	190	–0.04	–3.1	0.07	4.9	0.08
	[179, 201]		[–5.1, –0.7]		[3.0, 6.8]	
token bucket, burst 4	424	–0.12	–3.1	0.07	4.9	0.00
	[412, 437]		[–5.1, –0.7]		[2.8, 7.0]	
token bucket, burst 12	545	–0.11	–2.1	0.08	5.9	0.00
	[531, 559]		[–4.3, 0.2]		[3.3, 8.8]	

Table 9

User Harm Under a Full-Block Poisoning Attack.

Policy	Leaked (SMS)	Extra harm against attacked, no policy	
		all users	first-time users
none	1 976	–	–
sequential, default	1 357	1.09 [0.59, 1.59]	1.36
sequential T300, c0	858	2.55 [2.04, 3.07]	3.20
counter, 4 per 10 min	1 662	0.72 [0.44, 1.02]	0.91
counter + sequential	1 286	1.43 [0.88, 1.97]	1.79
token bucket, burst 4	1 877	0.12 [0.04, 0.23]	0.16
token bucket, burst 12	1 974	0.00 [0.00, 0.00]	0.00

messages sent, exempt, to real users’ trusted numbers are set aside, no counter run that started cold (44 of 100) exceeded Equation (3); three warm ones did, by 5 or 6 messages, and none exceeded Equation (4). With these unfitted inputs Equation (5) ordered the policies’ mean leakage in 18 of 18 decisive cells (two ties at N), and with the warm-start term in 17, missing where the estimates differ by 0.8 messages (30 blocks, 20 minutes; measured difference 6.6 [−27.2, 42.4]); per seed, 76 of 78 decisive comparisons agree. The algebraic crossing lies near five blocks for an hour and 30 for twenty minutes, where the nearest measured differences (three and 30 blocks) do not exclude zero; it is not a measured switching point. Paced to the quota, a pumper leaks about as much under the counter as with no policy (71 against 72 on three blocks in an hour), and 23 under the tests.

Against adaptive attackers (E3), the counter held the threshold-aware carrier, receipt faker, and concentrated trust builder to 24 to 36 messages (502 to 665 under the default), but not trust builders on random numbers (577, as under the default); over 360 minutes, the human-like carrier gained 81 messages an hour under the counter, and 753 under the default.

6.7. RQ5: what destination policies cost legitimate users

On one block, 20 000 Monte Carlo trials per cell give the probability of a verdict within 30 legitimate sends: 0.09% [0.06, 0.14] at 80% conversion with 20% autofill, 1.9% at 65% and 19.5% at 50%. Over 24 hours, the default raises 0.5 verdict events a day on 200 busy blocks at 80% conversion and 6.5 at 65%, where they meet 30 of about 28 800 requests (claim C4 held in 18 of 18 cells).

A poisoner flooding for 20 min the 20 blocks legitimate users concentrate on (caps lifted, ten seeds) raises 16 verdict events meeting 98 legitimate requests, 33 never completing; against observe-only operation on the same trace, enforcement costs 12.7 [4.6, 22.9] completions, 3.1% of users (6.3% without WhatsApp; 23.9% with a 24-hour denylist). One that stops after ten minutes leaves verdicts meeting 406.5 requests afterwards, an attributable loss of 46.2 [16.9, 79.8]. Grading limits the harm; it caps it neither at the challenge nor at the attack’s duration.

6.8. Adaptive attackers, economics and overhead

The leakages of these scripted adaptations are not bounds. Against v2 with caps lifted, a datacenter attacker buying every

challenge solution leaks 55, and a threshold-aware carrier and a receipt faker leak 570 each, drawing no verdict.

If pumpers earn a share of the 0.1422 USD retail price and pay an event-level bill, they break even at a share of 0.043 under v1 and 0.054 to 0.271 under v2 (the instant verifier 1.310); over 360 minutes under the counter, the human-like carrier needs 1.048. These are break-even shares for scripted attackers under assumed prices, not bounds or criminal profit.

On a real Redis (four vCPUs, four workers), sends cost 13.8 ms (median) and 21.1 ms (99th percentile), and 133 requests per second passed at concurrency 128 with the 400 ms floor and 50 ms vendor calls. With fast vendors, response times distinguished no pair of five outcome classes (smallest Kolmogorov–Smirnov $p = 0.051$). Under an adversarial mixture, 81% of sends overran the floor, and response-time thresholds separated sends from refusals with 88% balanced accuracy on the fitted run and, fitted on a new run, 95 to 99.9% on two others: a separation on that machine, not a deployment accuracy.

7. Discussion

In the scripted workloads, carriers that verify with human-like delay, fake receipts, or know the thresholds defeated our outcome tests but not a short-window counter. Within an hour of a verdict, the counter’s allowance, unlike the tests’, grows with the windows an attack spans; over longer campaigns, verdicts expire and the tests leak again. Equation (5) summarizes this for pumpers that solve no challenge and hold no exemption. Its inputs are in an operator’s logs; the cost of an excess send, competition under attack, and exemption abuse come only from the simulation. The counter and tests together, with a quota set from each block’s traffic, is a hypothesis for real traffic, not a recommendation.

8. Limitations

Everything was simulated, and calibration was assumed. Both protocols were written by the author for the author’s generator, the second knowing which counters had won. Some results follow from the generator (a perfect premium list, a datacenter attacker that declines challenges, returning users who always present verified history), and selection used an unweighted sum of four scripted pumpers’ leakage and a mean service target met on tuning seeds, which guarantees no service level elsewhere. Close rankings rest on ten or fewer seeds, and costs that round to zero are not shown equivalent. Per-client quotas within a block, carrier spend ceilings, bounded exemptions, tuned token buckets, and other sequential statistics lie outside the evaluated policies. Vendor adapters are untested live, and the timing result holds only for the observer tested. Destination policies challenge people for what strangers sharing their block did, and deployment under the GDPR or Saudi Arabia’s PDPL needs a lawful basis and a route for refused users.

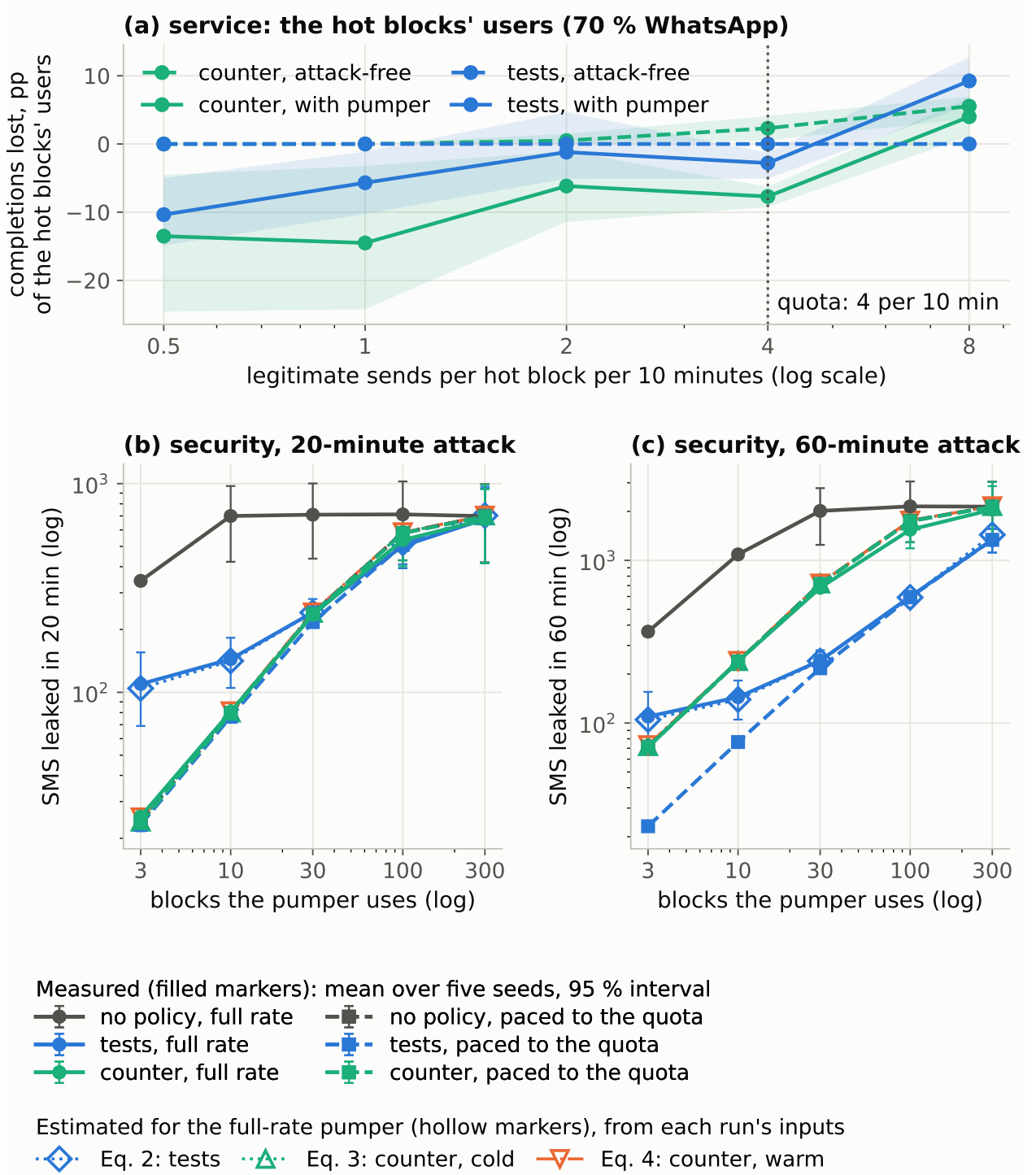


Fig. 3. Operating Boundary of the Counter.

9. Conclusions and future work

In this simulation, the design contains attacks that carry a signature at a low cost to simulated users and rations, rather than separates, residential attackers that resemble people. Counting sends on the destination leaked less in aggregate than our outcome tests against concentrated pumpers, although not reliably against an instantly verifying carrier; a frozen counter failed its

service claims when the attacker shared blocks with real users and lost its advantage against wide, paced, and trust-building pumpers. Within an hour, unfitted estimates ordered the policies' mean leakage in 18 of 18 decisive boundary cells with the cold-start bound and 17 with the warm-start bound, missing a near tie; the crossing was algebraic, not a measured switching point.

Replaying real traffic is the experiment this paper most needs:

it would test the assumed calibration, the cost of an excess send to users who share a block, and the quota a counter needs per block.

Declaration of generative AI and AI-assisted technologies in the writing process

The author used Claude (Anthropic) for code, evaluation, drafting and editing; checked numbers against the recorded results and claims against the code; and takes full responsibility for the content.

CRedit authorship contribution statement

Naveed Ahmed: Conceptualization, Methodology, Software, Validation, Investigation, Writing – original draft, Writing – review & editing.

Declaration of competing interest

The author declares no known competing financial interests or personal relationships that could have appeared to influence this work.

Data availability

No human or real traffic data was used. Code, protocols (config/), every run's configuration and outcome and the sources of headline numbers (results/) are at <https://github.com/nahmedpsu/OTP-DDOS>, release 2.9.1 (tag v2.9.1), which runs unchanged the code of release 2.9.0 (tag v2.9.0, commit 9af3100) that produced the results (code_identity.md); the 2.8.0 results are in results/historical_2.8. These files are the supplementary material.

References

Ahmed, N., 2026. OTP Guard: a layered SMS validation, risk-scoring and rate-limiting pipeline against OTP flood abuse (version 2.9.1). <https://github.com/nahmedpsu/OTP-DDOS>, tag v2.9.1. Results produced by 2.9.0 (commit 9af3100), whose executable code 2.9.1 keeps; 2.8.0 results (commit 1c277a0) in results/historical_2.8.

Commsrisk, 2023. Elon Musk says Twitter lost \$60mn a year because 390 telcos used bot accounts to pump A2P SMS. <https://commsrisk.com/elon-musk-says-twitter-lost-60mn-a-year-because-390-telcos-used-bot-accounts-to-pump-a2p-sms/>. Trade press; the figure is the company's own claim.

Dean, D., Stubblefield, A., 2001. Using client puzzles to protect TLS, in: 10th USENIX Security Symposium, USENIX Association.

Dionysiou, A., Athanasopoulos, E., 2020. SoK: Machine vs. machine – A systematic classification of automated machine learning-based CAPTCHA solvers. *Computers & Security* 97, 101947. <https://doi.org/10.1016/j.cose.2020.101947>.

Douceur, J.R., 2002. The Sybil attack, in: *Peer-to-Peer Systems (IPTPS 2002)*, Springer. pp. 251–260. https://doi.org/10.1007/3-540-45748-8_24.

Google LLC, 2026. SMS retriever API overview. <https://developers.google.com/identity/sms-retriever/overview>. Accessed 2026-10-01.

Heinaneen, J., Guerin, R., 1999. A single rate three color marker. RFC 2697, Internet Engineering Task Force. <https://doi.org/10.17487/RFC2697>.

Howard, S.R., Ramdas, A., McAuliffe, J., Sekhon, J., 2021. Time-uniform, nonparametric, nonasymptotic confidence sequences. *The Annals of Statistics* 49, 1055–1080. <https://doi.org/10.1214/20-AOS1991>.

Huh, J.H., Shin, H., Ahn, S., Yi, H., Cho, J., Kim, T., Lim, M., Choi, N., 2025. Preventing artificially inflated SMS attacks through large-scale traffic inspection, in: *34th USENIX Security Symposium*, USENIX Association, Seattle, WA.

Mi, X., Feng, X., Liao, X., Liu, B., Wang, X., Qian, F., Li, Z., Alrizah, S., Chen, L., Qian, Y., 2019. Resident evil: Understanding residential IP proxy as a dark service, in: *2019 IEEE Symposium on Security and Privacy (S&P)*, IEEE. <https://doi.org/10.1109/SP.2019.00011>.

MITRE Corporation, 2026. Resource hijacking: SMS pumping, technique T1496.003, MITRE ATT&CK. <https://attack.mitre.org/techniques/T1496/003/>. Accessed 2026-10-01.

Motoyama, M., Levchenko, K., Kanich, C., McCoy, D., Voelker, G.M., Savage, S., 2010. Re: CAPTCHAs – understanding CAPTCHA-solving services in an economic context, in: *19th USENIX Security Symposium*, USENIX Association.

Mulliner, C., Borgaonkar, R., Stewin, P., Seifert, J.P., 2013. SMS-based one-time passwords: Attacks and defense, in: *Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA 2013)*, Springer. pp. 150–159. https://doi.org/10.1007/978-3-642-39235-1_9.

NIST/SEMATECH, 2026. CUSUM control charts, section 6.3.2.3, e-handbook of statistical methods. <https://www.itl.nist.gov/div898/handbook/pmc/section3/pmc323.htm>. Accessed 2026-10-02.

Page, E.S., 1954. Continuous inspection schemes. *Biometrika* 41, 100–115. <https://doi.org/10.2307/2333009>.

- Richter, P., Wolsing, F., Chandrasekaran, A., Padmanabhan, R., Allman, M., Bush, R., Feldmann, A., 2016. A multi-perspective analysis of carrier-grade NAT deployment, in: Proceedings of the 2016 ACM Internet Measurement Conference (IMC), ACM. <https://doi.org/10.1145/2987443.2987474>.
- Sahin, M., Francillon, A., 2021. Understanding and detecting international revenue share fraud, in: Network and Distributed System Security Symposium (NDSS), Internet Society. <https://doi.org/10.14722/ndss.2021.24051>.
- Sahin, M., Francillon, A., Gupta, P., Ahamad, M., 2017. SoK: Fraud in telephony networks, in: 2017 IEEE European Symposium on Security and Privacy (EuroS&P), IEEE. <https://doi.org/10.1109/EuroSP.2017.40>.
- Siadati, H., Nguyen, T., Gupta, P., Jakobsson, M., Memon, N., 2017. Mind your SMSes: Mitigating social engineering in second factor authentication. *Computers & Security* 65, 14–28. <https://doi.org/10.1016/j.cose.2016.09.009>.
- Twilio Inc., 2026a. SMS pumping risk score (Lookup v2). <https://www.twilio.com/docs/lookup/v2-api/sms-pumping-risk>. Accessed 2026-10-01.
- Twilio Inc., 2026b. Verify API. <https://www.twilio.com/en-us/user-authentication-identity/verify>. Accessed 2026-10-03.
- Twilio Inc., 2026c. Verify fraud guard. <https://www.twilio.com/docs/verify/preventing-toll-fraud/sms-fraud-guard>. Accessed 2026-10-01.
- Veeravalli, V.V., Banerjee, T., 2012. Quickest change detection. arXiv:1210.5552 (book chapter).
- Wald, A., 1945. Sequential tests of statistical hypotheses. *The Annals of Mathematical Statistics* 16, 117–186. <https://doi.org/10.1214/aoms/1177731118>.
- Web Incubator Community Group, 2026. WebOTP API. <https://github.com/WICG/WebOTP>. Accessed 2026-10-01.
- Wiefling, S., Lo Iacono, L., Dürmuth, M., 2019. Is this really you? An empirical study on risk-based authentication applied in the wild, in: ICT Systems Security and Privacy Protection (IFIP SEC 2019), Springer. pp. 134–148. https://doi.org/10.1007/978-3-030-22312-0_10.
- Zaliskyi, M., Odarchenko, M., 2026. Artificially inflated traffic in A2P messaging: Window-based detection models and prefix-level analysis for fraud mitigation. *Science-Based Technologies* 69, 89–98. <https://doi.org/10.18372/2310-5461.69.20735>.